



Problem 1 - Problem Description

Problem 1: Pressure control of an FEA using Pulse Width Modulation (PWM) (100 points)

Figure 1 shows a pneumatic circuit used for controlling a Pneu-Net style Fluidic Elastomer Actuator (FEA). The curvature of the actuator is controlled by tuning the pressure in the actuator using the pneumatic circuit. The circuit consists of a pump supplying a constant pressure of 20 kPa, and a 3/2 spring-loaded solenoid valve to control the actuation. The pressure inside the actuator is controlled using Pulse Width Modulation (PWM; i.e. by turning the solenoid valve on and off rapidly, as discussed in the lecture).

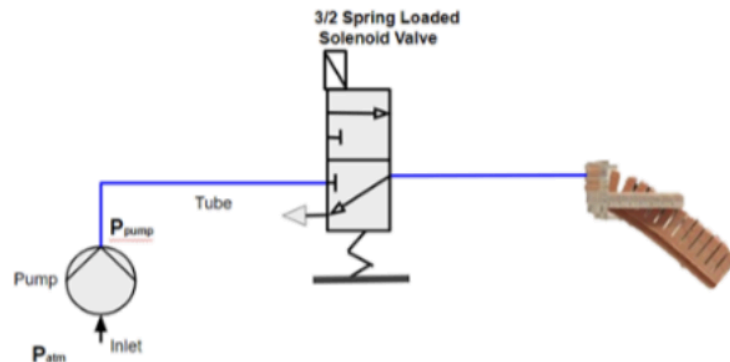


Figure 1: Design Parameters for the PneuNets Actuator (Figure not to Scale)

Part B - Actuator Pressure vs Time

```
In [1]: import numpy as np
import matplotlib.pyplot as plt

# --- System Parameters ---
P_pump = 20_000.0      # Pump pressure (Pa)
R_valve = 1.4e6        # Flow resistance (Pa·s / m³)
dp_per_mass = 1250.0 / 0.0025 # Pa per kg (from 1.25 kPa per 2.5 g)
rho_air = 1.225        # kg/m³, air density
C_act = 1 / (rho_air * dp_per_mass) # Actuator compliance (m³ / Pa)
tau = R_valve * C_act  # Time constant (s)

# --- PWM Settings ---
freq = 1.0             # Hz (switching frequency)
duty = 0.75            # 75% ON time
T = 1.0 / freq         # Period (s)
t_on = duty * T
t_off = T - t_on

# --- Simulation Parameters ---
t_final = 20.0         # total simulation time (s)
dt = 0.001            # time step (s)
n_steps = int(t_final / dt)
t = np.linspace(0, t_final, n_steps)
p = np.zeros(n_steps) # pressure array (Pa)

# --- Simulation Loop ---
for i in range(1, n_steps):
```

```

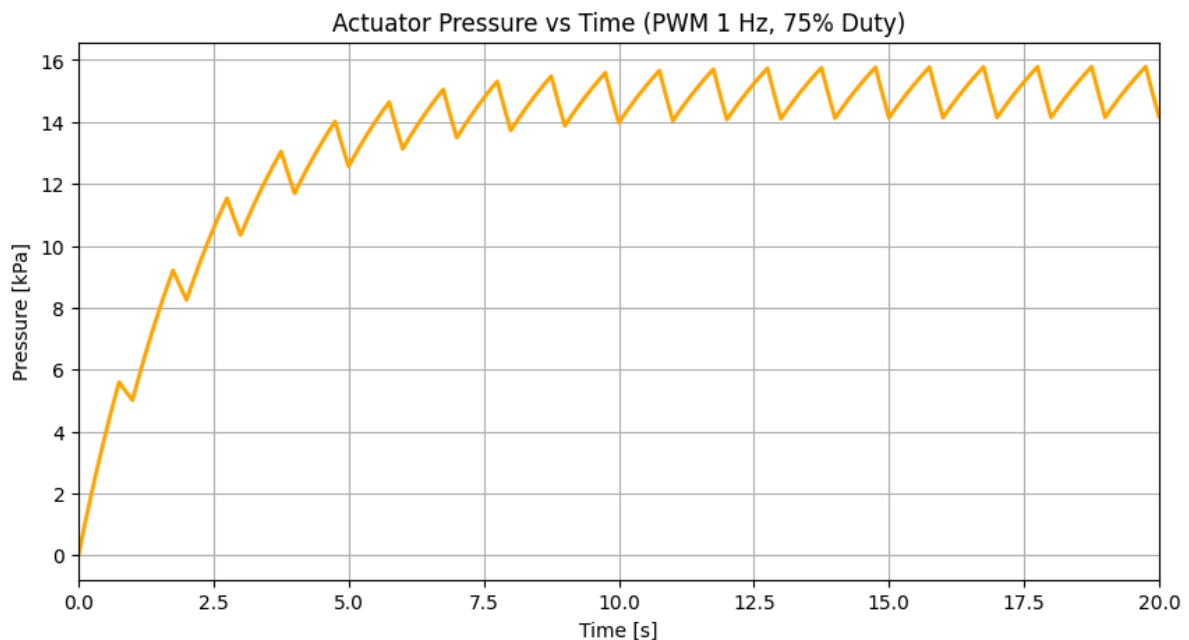
phase = t[i] % T # where we are in the PWM cycle
if phase < t_on: # valve ON (pressurizing)
    dpdt = (P_pump - p[i-1]) / (R_valve * C_act)
else:           # valve OFF (venting)
    dpdt = -p[i-1] / (R_valve * C_act)
p[i] = p[i-1] + dpdt * dt

# --- Convert and Plot ---
p_kpa = p / 1000 # convert Pa → kPa

plt.figure(figsize=(10, 5))
plt.plot(t, p_kpa, color='orange', lw=2)
plt.title("Actuator Pressure vs Time (PWM 1 Hz, 75% Duty)")
plt.xlabel("Time [s]")
plt.ylabel("Pressure [kPa]")
plt.grid(True)
plt.xlim(0, t_final)
plt.show()

# --- Optional Output ---
print(f"Actuator compliance: {C_act:.3e} m³/Pa")
print(f"Time constant (tau): {tau:.2f} s")
print(f"Steady-state pressure: {p_kpa[-1]:.2f} kPa")

```



Actuator compliance: 1.633e-06 m³/Pa

Time constant (tau): 2.29 s

Steady-state pressure: 14.16 kPa

Part C - Tip Displacement vs Time

```

In [20]: import numpy as np
import matplotlib.pyplot as plt

# --- System Parameters ---
P_pump = 20_000.0 # Pump pressure (Pa)
R_valve = 1.4e6 # Flow resistance (Pa·s / m³)
dp_per_mass = 1250.0 / 0.0025 # Pa per kg (from 1.25 kPa per 2.5 g)

```

```

rho_air = 1.225          # kg/m³, air density
C_act = 1 / (rho_air * dp_per_mass) # Actuator compliance (m³ / Pa)
tau = R_valve * C_act    # Time constant (s)

# --- PWM Settings ---
freq = 1.0              # Hz (switching frequency)
duty = .75              # 75% ON time
T = 1.0 / freq          # Period (s)
t_on = duty * T

# --- Geometric Parameters ---
hc = 0.014              # Channel height (m)
hb = 0.016              # Rubber height (m)
Lc = 0.012              # Channel Length (m)
L = 0.116               # Rubber Length (m)
E = 8.0e5               # Young's Modulus (Pa)
N = 7                   # Number of channels

# --- Simulation Parameters ---
t_final = 20.0          # total simulation time (s)
dt = 0.001              # time step (s)
n_steps = int(t_final / dt)
t = np.linspace(0, t_final, n_steps)
p = np.zeros(n_steps)   # pressure array (Pa)

# --- Simulation Loop ---
for i in range(1, n_steps):
    phase = t[i] % T    # where we are in the PWM cycle
    if phase < t_on:    # valve ON (pressurizing)
        dpdt = (P_pump - p[i-1]) / (R_valve * C_act)
    else:               # valve OFF (venting)
        dpdt = -p[i-1] / (R_valve * C_act)
    p[i] = p[i-1] + dpdt * dt

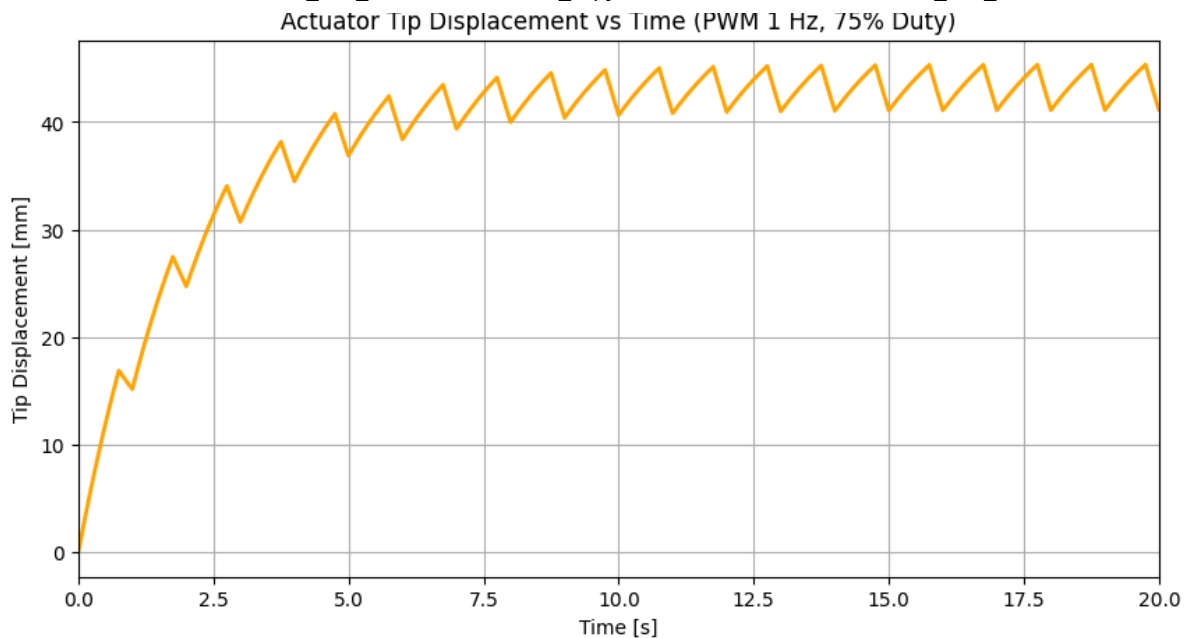
# --- Compute angle and displacement ---
theta = 2 * N * np.arctan(((Lc * p) / (2 * E)) * (1 / (hb - hc)))

# Avoid divide-by-zero when theta ~ 0 using small-angle approximation Δy ≈ L * θ
delta_y = np.zeros_like(theta)
small = np.abs(theta) < 1e-6
delta_y[small] = L * theta[small] / 2.0
delta_y[~small] = (L / theta[~small]) * (1 - np.cos(theta[~small]))

# --- Plot tip displacement over time ---
plt.figure(figsize=(10, 5))
plt.plot(t, delta_y * 1000, color='orange', lw=2) # convert m → mm
plt.title("Actuator Tip Displacement vs Time (PWM 1 Hz, 75% Duty)")
plt.xlabel("Time [s]")
plt.ylabel("Tip Displacement [mm]")
plt.grid(True)
plt.xlim(0, t_final)
plt.show()

print(f"Max tip displacement: {np.max(delta_y)*1000:.2f} mm")

```



Max tip displacement: 45.33 mm

Part D - Duty Cycle w/ 1.5cm as max displacement

```
In [32]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq

# --- System Parameters ---
P_pump = 20_000.0      # Pa
R_valve = 1.4e6        # Pa·s/m³
dp_per_mass = 1250.0 / 0.0025
rho_air = 1.225
C_act = 1 / (rho_air * dp_per_mass)
tau = R_valve * C_act

# --- PWM Settings ---
freq = 1.0             # Hz
T = 1.0 / freq

# --- Geometric Parameters ---
hc = 0.014
hb = 0.016
Lc = 0.012
L = 0.116
E = 8.0e5
N = 7

# --- Simulation Parameters ---
t_final = 50.0         # Long enough to reach steady state
dt = 0.001
n_steps = int(t_final / dt)
t = np.linspace(0, t_final, n_steps)

# Target tip displacement
target_dy = 0.015     # m

def simulate_pwm(duty):
```

```

t_on = duty * T
p = np.zeros(n_steps)

for i in range(1, n_steps):
    phase = t[i] % T
    if phase < t_on:
        dpdt = (P_pump - p[i-1]) / (R_valve * C_act)
    else:
        dpdt = -p[i-1] / (R_valve * C_act)
    p[i] = p[i-1] + dpdt * dt

theta = 2 * N * np.arctan((Lc * p) / (2 * E * (hb - hc)))
delta_y = np.zeros_like(theta)
small = np.abs(theta) < 1e-6
delta_y[small] = L * theta[small] / 2.0
delta_y[~small] = (L / theta[~small]) * (1 - np.cos(theta[~small]))

# Return steady-state displacement (average over last 5 seconds)
return np.mean(delta_y[-int(5/dt):])

# --- Define function to find root ---
def duty_to_target(duty):
    return simulate_pwm(duty) - target_dy

# --- Find duty cycle using brentq ---
duty_target = brentq(duty_to_target, 0.1, 0.9) # search between 10% and 90%
print(f"Approximate duty cycle for Δyss = {target_dy*1000:.1f} mm: {duty_target}")

# --- Plot the time response ---
delta_y = simulate_pwm(duty_target)

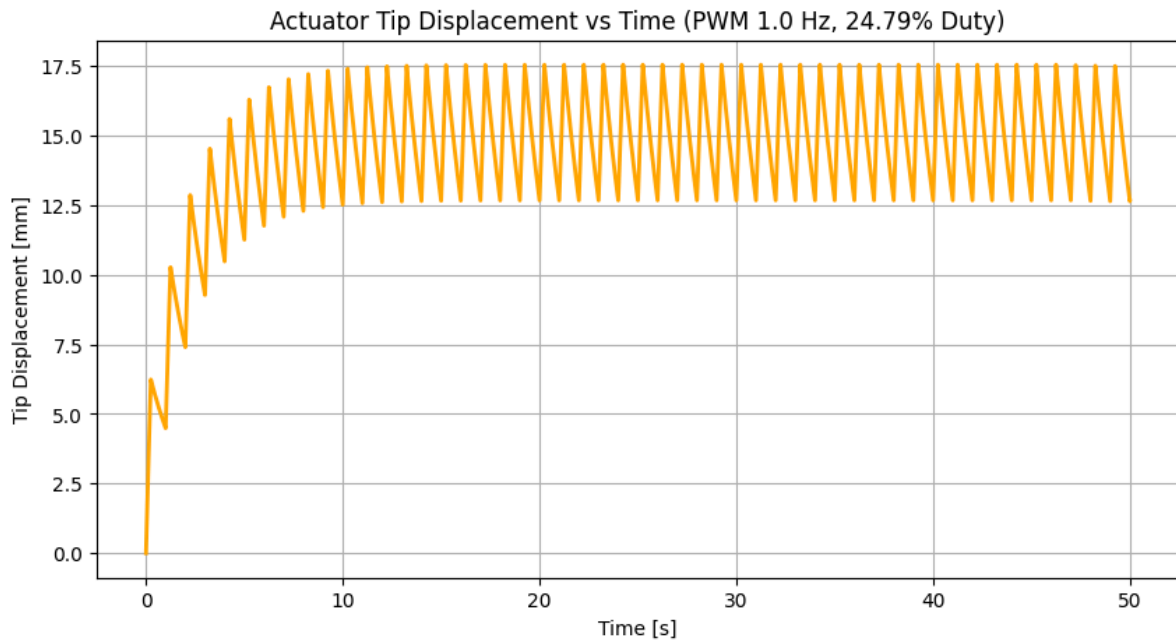
# For full time response
t_on = duty_target * T
p = np.zeros(n_steps)
for i in range(1, n_steps):
    phase = t[i] % T
    if phase < t_on:
        dpdt = (P_pump - p[i-1]) / (R_valve * C_act)
    else:
        dpdt = -p[i-1] / (R_valve * C_act)
    p[i] = p[i-1] + dpdt * dt

theta = 2 * N * np.arctan((Lc * p) / (2 * E * (hb - hc)))
delta_y = np.zeros_like(theta)
small = np.abs(theta) < 1e-6
delta_y[small] = L * theta[small] / 2.0
delta_y[~small] = (L / theta[~small]) * (1 - np.cos(theta[~small]))

plt.figure(figsize=(10,5))
plt.plot(t, delta_y*1000, color='orange', lw=2)
plt.title(f"Actuator Tip Displacement vs Time (PWM {freq} Hz, {duty_target*100:}%)")
plt.xlabel("Time [s]")
plt.ylabel("Tip Displacement [mm]")
plt.grid(True)
plt.show()

```

Approximate duty cycle for $\Delta y_{ss} = 15.0$ mm: 24.79%



Part E - Reduce Oscillations

```
In [33]: import numpy as np
import matplotlib.pyplot as plt

# --- System Parameters ---
P_pump = 20_000.0      # Pa
R_valve = 1.4e6        # Pa·s/m³
dp_per_mass = 1250.0 / 0.0025
rho_air = 1.225

# --- PWM Settings ---
freq = 1.0             # Hz
T = 1.0 / freq
duty = 0.75            # can use previous duty or find new for each if needed

# --- Geometric Parameters ---
hc = 0.014
hb = 0.016
Lc = 0.012
L = 0.116
E = 8.0e5
N = 7

# --- Simulation Parameters ---
t_final = 50.0
dt = 0.001
n_steps = int(t_final / dt)
t = np.linspace(0, t_final, n_steps)

# --- Different Actuator Compliances ---
C_list = [3 / (rho_air * dp_per_mass), 1 / (rho_air * dp_per_mass)]
labels = ['C_act = 3/(rho*dp_per_mass)', 'C_act = 1/(rho*dp_per_mass)']
colors = ['orange', 'blue']
```

```

plt.figure(figsize=(10,5))

for C_act, label, color in zip(C_list, labels, colors):
    p = np.zeros(n_steps)
    t_on = duty * T

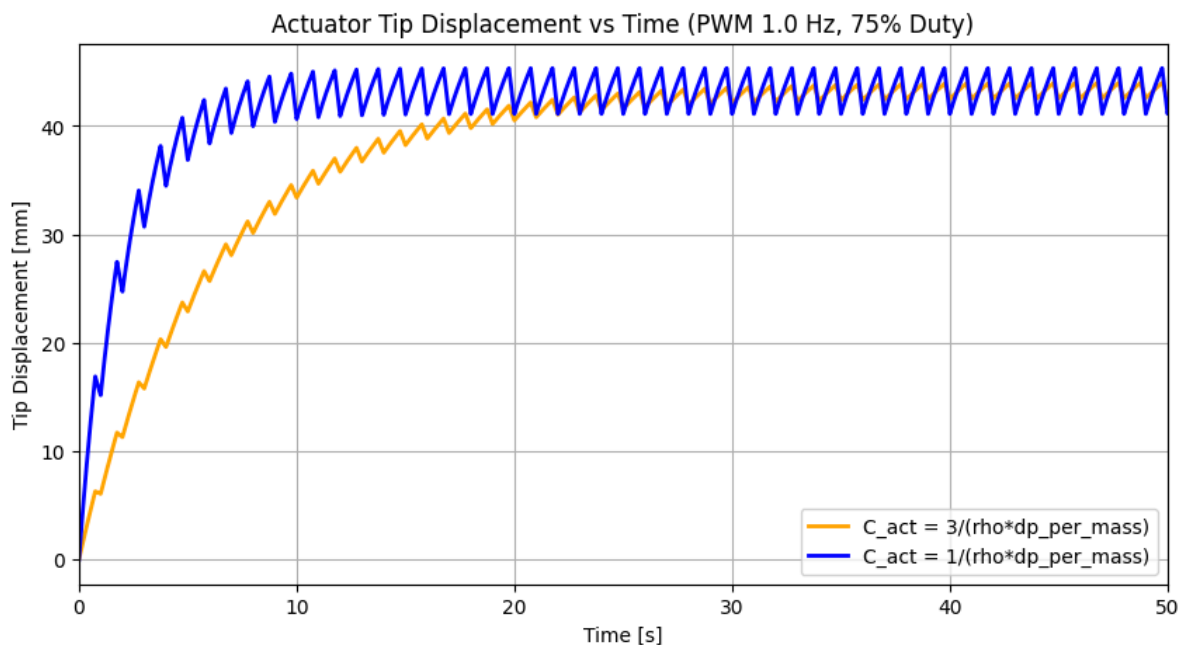
    for i in range(1, n_steps):
        phase = t[i] % T
        if phase < t_on:
            dpdt = (P_pump - p[i-1]) / (R_valve * C_act)
        else:
            dpdt = -p[i-1] / (R_valve * C_act)
        p[i] = p[i-1] + dpdt * dt

    # --- Convert pressure to tip displacement ---
    theta = 2 * N * np.arctan((Lc * p) / (2 * E * (hb - hc)))
    delta_y = np.zeros_like(theta)
    small = np.abs(theta) < 1e-6
    delta_y[small] = L * theta[small] / 2.0
    delta_y[~small] = (L / theta[~small]) * (1 - np.cos(theta[~small]))

    plt.plot(t, delta_y*1000, color=color, lw=2, label=label)

plt.title(f"Actuator Tip Displacement vs Time (PWM {freq} Hz, {duty*100:.0f}% D")
plt.xlabel("Time [s]")
plt.ylabel("Tip Displacement [mm]")
plt.grid(True)
plt.legend()
plt.xlim(0, t_final)
plt.show()

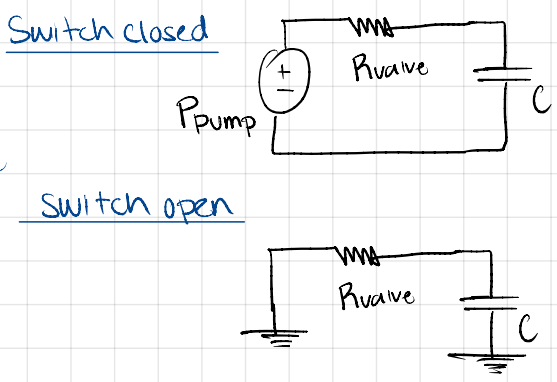
```



Part A

electrical - pneumatic analogy

i current	m mass flow rate
v voltage	p pressure
C elec. capacitance	C pneumatic capacitance



Part B

Apply KVL & KCL $V_{in} = V_R + V_C$ $V_R = iR$ $i = C \frac{dV_C}{dt}$

$$\Rightarrow V_{in} = RC \frac{dV_C}{dt} + V_C \quad @ V_{in} = 0 \quad \& \quad @ V_{in} \neq 0$$

@ $V_{in} = 0$ $0 = RC \frac{dV_C}{dt} + V_C \Rightarrow \int \frac{dV_C}{V_C} = \int \frac{-dt}{RC} = \ln|V_C| = \frac{-t}{RC} + \text{const} \Rightarrow V_C = V_0 e^{-t/RC}$ pump off

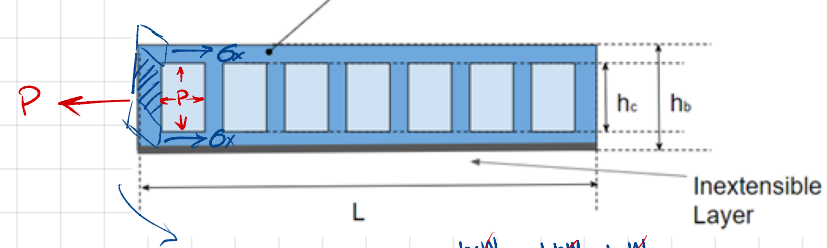
@ $V_{in} = \text{const}$ $RC \frac{dV_C}{dt} + V_C = V_{in} \Rightarrow \frac{dV_C}{dt} + \frac{1}{RC} V_C = \frac{1}{RC} V_{in}$ $\frac{dy}{dx} + p(x)y = q(x)$ $I.F = e^{\int p(x)dx} = e^{t/RC} = u(x)$ $y = \frac{1}{u(x)} \int u(x) \cdot q(x) dx$

$$\Rightarrow V_C(t) = e^{-t/RC} = e^{t/RC} \quad V_C(t) = \frac{1}{u(t)} \int_0^t u(t) \frac{V_{in}}{RC} dt = e^{-t/RC} \int_0^t e^{t/RC} \frac{V_{in}}{RC} dt = e^{-t/RC} \left(RC e^{t/RC} \frac{V_{in}}{RC} + C_1 \right) \Big|_0^t$$

$$\Rightarrow e^{-t/RC} \left(e^{t/RC} V_{in} + C_1 \right) - (V_{in} + C_1) \Rightarrow \text{Apply } V_C(0) = V_0 e^{-t/RC} \quad V_C(t) = (V_0 - V_{in}) e^{-t/RC} + V_{in}$$
 pump on

Part C

Silicone, $E = 0.8 \text{ MPa}$

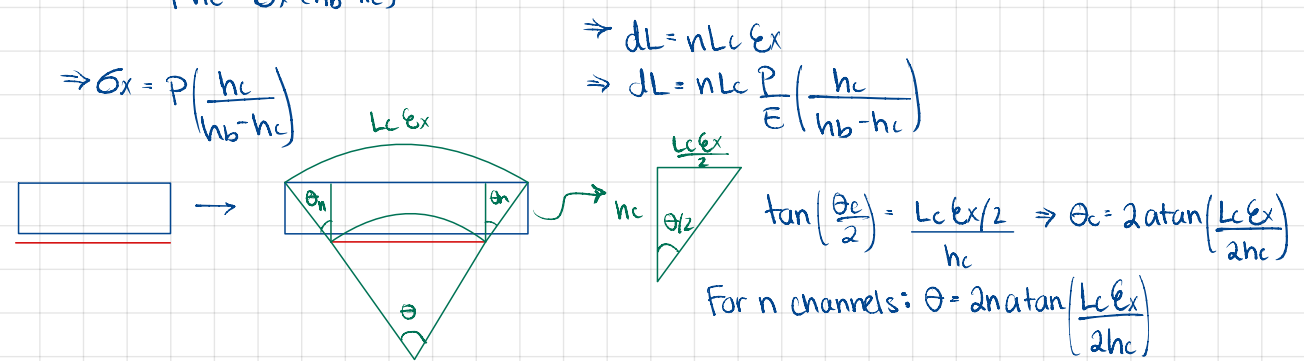


- ### Assumptions
- * only axial stress
 - * uniform stress distribution
 - * linear elasticity

Given Force balance $P A_c = \sigma_x (A_b - A_c)$ Hook's Law: $\sigma_x = E \epsilon_x$ for n channels:

$$P h_c = \sigma_x (h_b - h_c)$$

- $h_c = 1.4 \text{ cm}$
- $h_b = 1.6 \text{ cm}$
- $L_{\text{channel}} = 1.2 \text{ cm}$
- $N_{\text{channel}} = 7$
- $L = 11.6 \text{ cm}$
- $E = 0.8 \text{ MPa}$



$\Rightarrow$ Arc length: $L = r \theta$ Curvature: $k = \frac{1}{r} = \frac{\theta}{L}$

$$\Rightarrow \delta_y = r - r \cos \theta = r(1 - \cos \theta) \Rightarrow \delta_y = \frac{L}{\theta} (1 - \cos \theta)$$

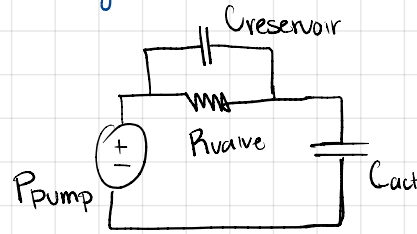
$$\Rightarrow \epsilon_x = \frac{P}{E} \left(\frac{h_c}{h_b - h_c} \right) \quad \theta = 2 n \arctan\left(\frac{L}{2 h_c} \left(\frac{P}{E} \left(\frac{h_c}{h_b - h_c} \right) \right)\right)$$

$$= 2 n \arctan\left(\frac{L P}{2 E} \left(\frac{1}{h_b - h_c} \right)\right)$$

Part E In order to reduce fluctuations we can add an extra air reservoir in parallel:

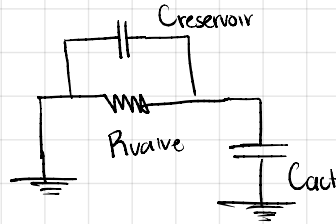
More volume $\rightarrow$ slower pressure changes $\rightarrow$ smaller fluctuations

Switch closed



$$C_{total} = C_{reservoir} + C_{act}$$

Switch open



We can assume $C_{reservoir} = 2C$. So $C_{total} = 3C$ lets input this value in the code $\therefore$ we change the following:

$$C_{act} = \frac{\text{OLD}}{p(dp-per-mass)} = \frac{1}{p(dp-per-mass)}$$

$$C_{act} = \frac{\text{New}}{p(dp-per-mass)} = \frac{3}{p(dp-per-mass)}$$

Adding the parallel reservoir, makes the actuator respond more slowly to the on/off switching of the PWM valve, which filters high frequency pressure oscillations and reducing fluctuations in tip displacement